Introduction to Computer Systems

Lecture 7: Concurrency and Synchronization (2)

Course Name: Introduction to Computer Systems
Instructors: Wei Xu, Mingyu Gao
Teaching Assistants: Weitao Wan, Tingrui Zhang, Ruitao Liu, Shuo Chen

April 12, 2026

Contents

1 Introduction **3**

1.1 Brief Recap: Locks, Semaphores, and Monitors . . . 3

2 The Readers–Writers Problem **4**

2.1 Motivation and Problem Statement . . . 4

2.2 Correctness Constraints . . . 5

2.2.1 Safety . . . 5

2.2.2 Liveness . . . 5

2.2.3 Performance . . . 5

2.3 Monitor State and Policy Variables . . . 5

2.4 Reader Algorithm . . . 6

2.5 Writer Algorithm . . . 7

2.6 Why the Algorithms Are Correct . . . 7

2.7 A Worked Execution Trace . . . 8

2.8 Design Questions and Variants . . . 8

2.8.1 What if readers ignore waiting writers? . . . 8

2.8.2 Why signal one writer but broadcast to readers? . . . 9

2.8.3 What if `signal` becomes `broadcast` when the last reader exits? . . . 9

2.8.4 Why use two condition variables instead of one? . . . 9

2.9 Mesa Monitors and Why We Must Use `while` . . . 10

3 Wakeups, Blocking, and the Cost of Thread Switching **10**

4 Programming Language Support for Synchronization **11**

4.1 Why Explicit Locking Is Fragile in C . . . 11

4.2 Exceptions Make the Problem Worse . . . 11

4.3 RAII and the Synchronized Destructor Pattern in C++ . . . 12

4.4 The KISS Principle and Long-Lived Code . . . 12

4.5 Java's Built-In Monitor Support . . . 12

5 Shared Memory versus Message Passing **13**

5.1 Two Different Styles of Coordination . . . 13

5.2 A Simple Go Example . . . 14

5.3 Equivalent Expressive Power, Different Programming Style . . . 14

6 Concurrency Bugs in Practice **15**

7 Resource Contention and Deadlocks **15**

7.1 Resources: Preemptable and Non-Preemptable 15

7.2 Starvation versus Deadlock 15

7.3 Deadlocks Are Often Nondeterministic 16

7.4 The Four Necessary Conditions 16

7.5 Case Study: Bridge Crossing 16

7.6 Case Study: Dining Philosophers 17

7.7 Detection and Recovery 17

7.8 Prevention, Avoidance, and Practical Operating Systems 18

7.8.1 Prevention 18

7.8.2 Avoidance 19

7.8.3 Ignoring the Problem 19

8 Behind the Scenes: Implementing a Lock **19**

8.1 The Naive Uniprocessor Idea: Disable Interrupts 19

8.2 A Better Interrupt-Based Design 20

8.3 Atomic Read–Modify–Write Instructions 20

8.4 A Simple Spinlock 20

8.5 Reducing Busy Waiting 20

8.6 Comparison with the Interrupt-Based Solution 21

9 Summary **21**

10 Quizzes & Exercises **22**

10.1 Short-Answer Quiz 22

10.2 Concise Answer Key 22

10.3 Essay Questions 23

10.4 Suggested Analytical Exercises 23

1 Introduction

This lecture develops the second half of the course's treatment of concurrency. The main theme is that synchronization is not merely about preventing two threads from touching the same variable at the same instant. Rather, it is about expressing *collaboration policies*: who may proceed, who must wait, who should be awakened, and how such decisions can be made without violating correctness.

The lecture proceeds in four layers. First, it studies the readers–writers problem as a full monitor-based synchronization design, including both the algorithm and the policy choices hidden behind the code. Second, it explains why modern programming languages provide explicit support for synchronization, rather than forcing programmers to write every lock acquisition and release by hand. Third, it broadens the discussion from shared-memory synchronization to message passing and then to empirical observations about real concurrency bugs. Finally, it turns to resource contention and deadlock, concluding with a brief systems-level look at how locks can actually be implemented.

1.1 Brief Recap: Locks, Semaphores, and Monitors

A lock provides the most direct implementation of mutual exclusion. One thread calls `Acquire()`, enters the critical section, and later calls `Release()`. If another thread tries to acquire the same lock while it is busy, the second thread must wait. This simple interface hides an important requirement: the acquire and release operations themselves must be implemented atomically. If two waiting threads simultaneously observe a lock as free and both succeed, mutual exclusion is lost immediately.

Semaphores generalize the lock idea by replacing the boolean locked/unlocked state with a nonnegative integer counter. The `P()` operation waits for the counter to become positive and then decrements it; the `V()` operation increments the counter and wakes a waiter if necessary. In practice, semaphores are used in two quite different ways:

- **Mutual exclusion**: a semaphore initialized to 1 acts like a mutex.
- **Scheduling constraints**: a semaphore can count available buffer slots, produced items, or other quantities that control when a thread is allowed to continue.

This dual use is powerful, but it also makes semaphore code harder to reason about. The same primitive is simultaneously carrying "who owns the shared object right now" and "whether the precondition for progress is satisfied." Monitors were introduced to separate these two concerns more cleanly.

A monitor combines:

- a lock, which protects the shared state and enforces mutual exclusion, and
- condition variables, which provide named waiting places for threads that cannot yet proceed.

The key monitor operation is `wait(condition, lock)`. It must atomically release the lock and put the thread to sleep. When the thread wakes up, it reacquires the lock before returning from the wait. That single atomic step is what allows a thread to sleep *inside* a critical section without freezing the entire system.

Deep Dive

The conceptual gain of a monitor is not merely syntactic. In a semaphore solution, mutual exclusion and scheduling are interleaved in one stream of `P()` and `V()` calls. In a monitor solution, the code structure itself mirrors the underlying logic: the lock protects the shared data structure, while condition variables express when particular operations are allowed to proceed. That separation is a major aid to correctness, maintenance, and proof.

2 The Readers–Writers Problem

2.1 Motivation and Problem Statement

Consider a shared database or other large shared object used by two classes of threads:

- **Readers**, which only inspect the state and never modify it.
- **Writers**, which both read and modify the state.

A single giant mutex around the whole database would certainly be correct, but it would be unnecessarily restrictive. Multiple readers can safely coexist, because read-only operations do not interfere with one another. Writers are different: while a write is in progress, no other writer may write, and no reader should observe a partially updated state.

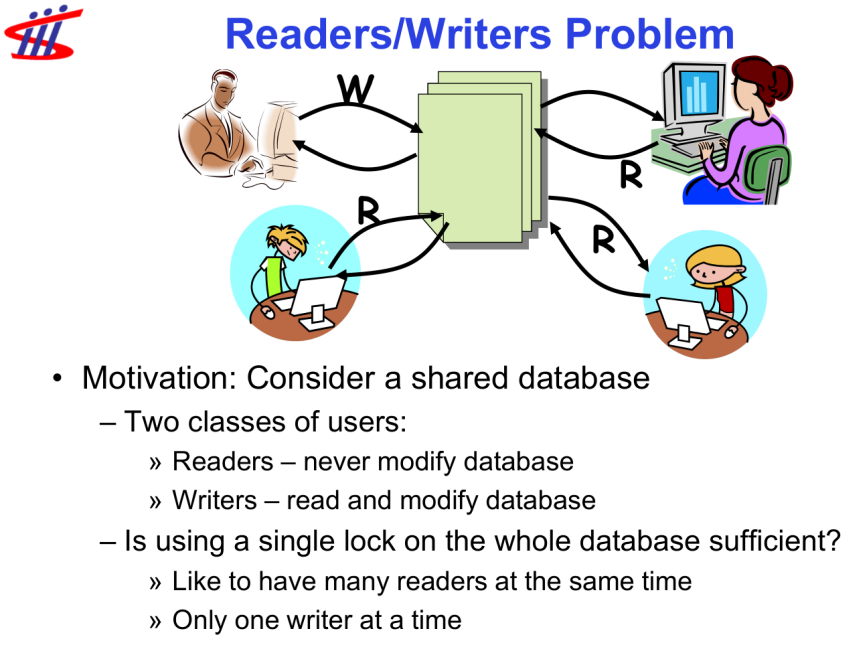


Figure 1: Motivating view of the readers–writers problem. A shared database is accessed by multiple readers and writers. The desired policy is not ordinary all-or-nothing mutual exclusion; it is a mixed policy in which many readers may proceed concurrently, but any writer must receive exclusive access to the shared state.

This problem appears everywhere in systems. File system metadata, in-memory indexes, caches, routing tables, page tables, and database records all have phases in which reads are frequent but

writes are comparatively rare. The design challenge is therefore not only correctness but also concurrency: the synchronization policy should preserve as much safe parallelism as possible.

2.2 Correctness Constraints

A correct readers–writers solution must satisfy three classes of requirements.

2.2.1 Safety

1. Readers may access the database only when there is no active writer.
2. A writer may access the database only when there are no active readers and no active writers.
3. The bookkeeping variables used to decide who may enter must themselves be manipulated atomically.

2.2.2 Liveness

A correct design should also avoid avoidable starvation. In particular, if writers are ignored for too long, a constant stream of incoming readers can indefinitely delay a waiting writer. Conversely, if writers receive absolute priority, readers can experience long delays in a write-heavy workload.

2.2.3 Performance

The synchronization policy should not serialize operations unnecessarily. In particular, the *actual* database access should not occur while the monitor lock is held if doing so would destroy legitimate reader concurrency.

2.3 Monitor State and Policy Variables

We can use a monitor with the following protected state:

`AR`	Number of active readers currently inside the database
`WR`	Number of waiting readers sleeping on the reader condition variable
`AW`	Number of active writers currently inside the database (in a correct design this is either 0 or 1)
`WW`	Number of waiting writers sleeping on the writer condition variable
`okToRead`	Condition variable for waiting readers
`okToWrite`	Condition variable for waiting writers

These variables are all protected by a single monitor lock. The algorithm studied here is a **writer-priority** design: once a writer is waiting, newly arriving readers are no longer admitted until the writers have had a chance to proceed. This is why readers test not only whether a writer is *active*, but also whether any writer is *waiting*.

Definition

A **writer-priority readers–writers policy** allows multiple concurrent readers only when no writer is active and no writer is already queued for service. It sacrifices some read throughput in exchange for preventing writer starvation.

2.4 Reader Algorithm

Algorithm 1: Reader Entry and Exit under Writer Priority

Input: Monitor state `AR`, `WR`, `AW`, `WW`; lock; conditions `okToRead`, `okToWrite`
Output: One completed read that respects writer-priority synchronization

```
1  Acquire lock;
2  while AW + WW > 0 do
3      WR = WR + 1;
4      Wait on okToRead with lock;
5      WR = WR - 1;
6  AR = AR + 1;
7  Release lock;
8  Perform the read-only database access;
9  Acquire lock;
10 AR = AR - 1;
11 if AR == 0 and WW > 0 then
12     Signal okToWrite;
13 Release lock;
14 return success;
```

The reader performs two small critical sections. The first is the *check-in* phase: it decides whether the reader may enter immediately or must wait. The second is the *check-out* phase: after the read completes, the reader updates the counters and possibly wakes a writer.

This split is essential. If the monitor lock were held across the entire database access, the solution would degenerate into a single global mutex and no longer allow multiple readers to run concurrently. The lock is therefore held only while manipulating the synchronization state, not during the possibly long-running read itself.

2.5 Writer Algorithm

Algorithm 2: Writer Entry and Exit under Writer Priority

Input: Monitor state `AR`, `WR`, `AW`, `WW`; lock; conditions `okToRead`, `okToWrite`
Output: One completed write with exclusive access to the database

```
Acquire lock;
while AW + AR > 0 do
    WW = WW + 1;
    Wait on okToWrite with lock;
    WW = WW - 1;
AW = AW + 1;
Release lock;
Perform the read-write database access;
Acquire lock;
AW = AW - 1;
if WW > 0 then
    Signal okToWrite;
else if WR > 0 then
    Broadcast okToRead;
Release lock;
return success;
```

The writer's entry condition is simpler: it may proceed only when there are no active readers and no active writers. On exit, however, the writer must make a policy decision. If writers are waiting, it wakes exactly one writer. Otherwise, it wakes *all* waiting readers, since many readers can proceed together.

2.6 Why the Algorithms Are Correct

A full formal proof is beyond the scope, but the central invariants are straightforward.

Deep Dive

For the monitor state above, the critical invariants are:

$$AW \in \{0, 1\}, \qquad AR \geq 0, \qquad WR \geq 0, \qquad WW \geq 0,$$

$$AR > 0 \Longrightarrow AW = 0,$$

$$AW = 1 \Longrightarrow AR = 0.$$

The entry conditions preserve these invariants because a reader enters only when $AW = 0$ and $WW = 0$, while a writer enters only when both $AR = 0$ and $AW = 0$. The lock ensures that the counters are updated atomically, so no two threads can simultaneously decide to violate the policy.

Observe also the role of the waiting counters:

- `WW` does not merely record bookkeeping information. It actively *changes policy* by preventing new readers from bypassing waiting writers.
- `WR` allows the writer on exit to know whether a broadcast to readers is useful.

The condition variables are not boolean flags. They are waiting queues. The actual truth of the synchronization conditions is still represented by the counters, which is why every wait is enclosed in a `while` loop rather than an `if` statement.

2.7 A Worked Execution Trace

Let's simulate the sequence R_1, R_2, W_1, R_3, which is an excellent demonstration of writer priority. Initially,

$$AR = WR = AW = WW = 0.$$

The execution proceeds as follows:

Step	Event	Explanation	State After Event
1	R_1 arrives	No active or waiting writer exists, so R_1 enters immediately.	$AR = 1$
2	R_2 arrives	Readers do not block one another, so R_2 also enters.	$AR = 2$
3	W_1 arrives	A writer cannot enter while readers are active, so W_1 sleeps on `okToWrite`.	$AR = 2, WW = 1$
4	R_3 arrives	Because a writer is already waiting, writer priority prevents R_3 from joining the active readers. It sleeps on `okToRead`.	$AR = 2, WR = 1, WW = 1$
5	R_2 exits	It is not the last active reader, so no writer is awakened yet.	$AR = 1, WR = 1, WW = 1$
6	R_1 exits	Now AR becomes 0, so the last reader wakes one waiting writer.	$AR = 0, WR = 1, WW = 1$
7	W_1 runs	The writer wakes, rechecks its condition, enters, and writes.	$AW = 1, WR = 1$
8	W_1 exits	No writers remain waiting, so the writer broadcasts to the readers.	$AW = 0, WR = 1, WW = 0$
9	R_3 runs	R_3 wakes, rechecks, and enters as a normal reader.	$AR = 1$

This trace exposes the exact semantic role of writer priority. Without the `WW` term in the reader's waiting condition, R_3 would have been allowed to enter immediately. If more readers kept arriving, a writer could be postponed indefinitely.

2.8 Design Questions and Variants

2.8.1 What if readers ignore waiting writers?

Suppose the reader waited only while `AW > 0` and ignored `WW`. Then newly arriving readers would be allowed to enter so long as no writer had *yet* become active. In a read-heavy workload, this creates a classic starvation scenario: every time the last active reader is about to leave, another reader arrives and joins the set of active readers before the writer can begin.

This alternative policy is sometimes called a *reader-preference* design. It can deliver higher aggregate read throughput, but it provides weaker fairness guarantees for writers.

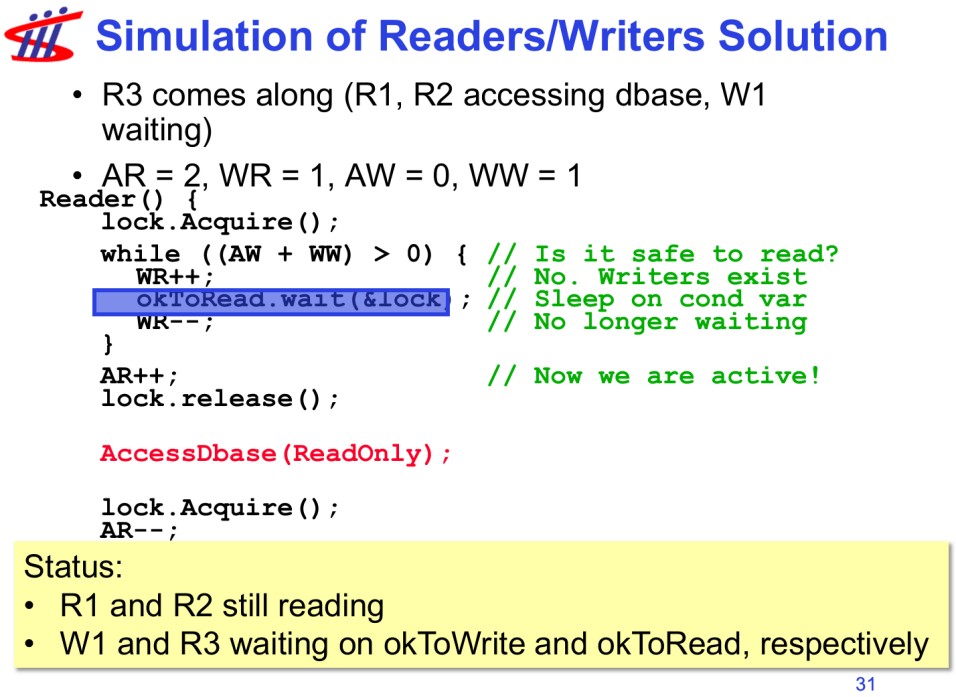


Figure 2: A representative intermediate state of the simulation. Two readers are still active, one writer is waiting to write, and a newly arriving reader is forced to wait because writer priority forbids later readers from overtaking a queued writer. This figure captures the core policy choice of the algorithm.

2.8.2 Why signal one writer but broadcast to readers?

At most one writer can be active, so signaling a single writer is sufficient. Waking every writer would be correct if each writer rechecked the condition in a `while` loop, but it would create unnecessary wakeups and scheduler overhead. By contrast, if no writer is waiting, all sleeping readers may safely wake and compete to become active readers together. A broadcast therefore preserves the main advantage of reader concurrency.

2.8.3 What if `signal` becomes `broadcast` when the last reader exits?

This change typically preserves safety but harms efficiency. If many writers are awakened simultaneously, all but one will merely rediscover that the condition is still false and go back to sleep. This is the familiar *thundering herd* problem: correctness survives, but the implementation performs needless context switches and lock acquisitions.

2.8.4 Why use two condition variables instead of one?

A single condition variable is tempting because both readers and writers are, in some sense, "waiting to continue." However, separate queues encode useful policy information. If the last reader exits and merely wakes an arbitrary sleeper from a single mixed queue, it may wake a reader when a writer should run next. That reader may discover that a writer is waiting and go back to sleep, while the writer remains asleep as well. The system can then reach a state in which nobody is left to make progress. Separate condition variables avoid this confusion by allowing the monitor to wake a *specific class* of thread.

2.9 Mesa Monitors and Why We Must Use `while`

The reader and writer algorithms both use `while` loops around their waits. This is not accidental boilerplate; it reflects the semantics of real monitor implementations.

Definition

In a **Hoare-style monitor**, a signal immediately transfers the monitor lock and execution control to the waiting thread. In a **Mesa-style monitor**, a signal merely moves the waiting thread to the runnable queue; the signaled thread must later compete to reacquire the lock.

Real operating systems overwhelmingly implement Mesa-style behavior because it fits naturally with an ordinary scheduler and ready queue. The price is that a signaled thread cannot assume the condition is still true when it finally runs. Another thread may have intervened and consumed the very condition that justified the signal.

For example, suppose a consumer is waiting because a queue is empty. A producer enqueues one item and signals the consumer. Before the consumer runs again, a different consumer may wake, reacquire the lock first, and remove that item. By the time the original waiter resumes, the queue is empty once more. Therefore the correct pattern is always:

Listing 1: Condition Variables Require Rechecking the Predicate

```
lock.acquire();
while (queue.isEmpty()) {
    dataready.wait(&lock);
}
item = queue.dequeue();
lock.release();
```

Using `if` instead of `while` is correct only under the much stronger Hoare-style semantics, which are elegant theoretically but unusual in production systems.

3 Wakeups, Blocking, and the Cost of Thread Switching

Some people often overestimate the performance significance of a few extra wakeups or condition-variable broadcasts and underestimate the complexity costs of clever synchronization code.

Whenever a thread blocks, wakes, or changes between runnable and sleeping states, the operating system is usually involved. This means that synchronization is tied to the scheduler and to kernel-maintained thread state. Historically, this overhead was large enough that user-level thread packages were considered highly attractive. In older systems, a kernel thread transition could be expensive enough that programmers worked hard to avoid it.

Modern systems are different. Kernel support for threads is now much faster than it once was, and the basic act of a context switch is often not the dominant cost. The more significant hidden cost is usually the disruption of caches, translation lookaside buffers, and memory locality. A thread switch within one address space and over a small shared working set may be very cheap; a switch to a thread with a completely different working set may be expensive because it triggers many subsequent cache misses.

The practical lesson is conservative but important: do not introduce complicated synchronization protocols merely to avoid a small number of wakeups or broadcasts unless you have real

evidence that they are the bottleneck. Over-optimized synchronization frequently becomes less readable, harder to verify, and more vulnerable to deadlock or atomicity bugs.

Deep Dive

Many synchronization bugs originate not in the first obviously wrong solution, but in the "clever" revision written to save a few context switches or eliminate a broadcast. In concurrent code, simplicity is often a performance optimization in disguise because simple code is easier to keep correct, easier to maintain, and less likely to create debugging disasters.

4 Programming Language Support for Synchronization

4.1 Why Explicit Locking Is Fragile in C

In a low-level language such as C, lock acquisition and lock release are completely explicit. This creates a familiar hazard: every possible control-flow path that exits the critical section must also release the lock. A function that appears simple on the normal path may still be wrong because an error path returns early.

Listing 2: An Explicit-Locking Bug Caused by an Early Return

```
int Rtn() {
    acquire(&lock);
    /* ... */
    int error = open("file_name");
    if (error) {
        return errReturnCode;    // lock is never released
    }
    /* ... */
    release(&lock);
    return OK;
}
```

The immediate repair is equally clear and equally fragile: release the lock before every return. But as the number of error paths grows, so does the chance that one path will be forgotten. Worse still, the bug may remain latent for a long time if the failing path is rare.

4.2 Exceptions Make the Problem Worse

Languages with exceptions, such as C++, create many additional exit paths. The danger is no longer limited to explicit `return` statements. If a function inside a critical section throws an exception, control may unwind past the lock-release point without executing it.

Listing 3: Exceptions Create Hidden Exit Paths from a Critical Section

```
void DoFoo();

void Rtn() {
    lock.acquire();
    /* ... */
    DoFoo();       // may throw
    /* ... */
    lock.release();
```

```
}
```

One possible fix is to wrap the critical section in a catch-all handler, release the lock, and then rethrow the exception. While correct in principle, this is clumsy and error-prone as a general programming style.

4.3 RAII and the Synchronized Destructor Pattern in C++

C++ solves this elegantly through scope-based resource management, now commonly summarized as RAII (*Resource Acquisition Is Initialization*). A small helper object acquires the lock in its constructor and releases it in its destructor. Because destructors execute automatically when control leaves the scope—whether through normal return or exception unwinding—the lock is released reliably.

Listing 4: Scope-Based Lock Management in C++

```
class lock {
    mutex &m_;
public:
    lock(mutex &m) : m_(m) {
        m_.acquire();
    }
    ~lock() {
        m_.release();
    }
};

mutex m;

void critical_section() {
    lock mylock(m);    // acquire here
    /* ... protected work ... */
}                      // automatic release here
```

This style is not merely a convenience. It encodes an invariant directly into the language's object-lifetime rules: if the scope ends, the resource is released. That removes a whole class of human error.

4.4 The KISS Principle and Long-Lived Code

The UNIX philosophy stresses a systems design principle that applies with unusual force to synchronization: **Keep It Simple, Stupid**. Explicit locks can be made efficient, but they are difficult to analyze because all invariants must continue to hold not only in the original version of the code but also after every future modification. Concurrent code often outlives the expectations of its original authors by many years. A simple synchronization discipline therefore has a large long-term payoff.

4.5 Java's Built-In Monitor Support

Java takes a different but equally important approach: it integrates monitors directly into the language. Every object has an intrinsic monitor. A `synchronized` instance method implicitly acquires the monitor associated with `this`. The programmer does not manually write lock

acquisition at the start and lock release at the end of the method; the language guarantees those semantics.

Listing 5: A Java Synchronized Queue Using an Object Monitor

```java
public class SynchronizedQueue {
    public LinkedList<Integer> q = new LinkedList<Integer>();

    public synchronized void enqueue(int item) {
        q.add(item);
        notify();
    }

    public synchronized int dequeue() {
        try {
            while (q.size() == 0) {
                wait();
            }
            return q.removeFirst();
        } catch (InterruptedException e) {
            return 0;
        }
    }
}
```

Two questions are pedagogically important here.

1. **Where is the lock?** The lock is not a separate user-defined variable in the code; it is the intrinsic monitor attached to the queue object.
2. **What is the scope of the lock?** For an instance method, the scope is the particular object on which the method is invoked. Different objects of the same class have different monitors.

This language-level support makes synchronization more uniform, reduces boilerplate, and helps ensure that exceptions do not silently leave locks held.

5 Shared Memory versus Message Passing

5.1 Two Different Styles of Coordination

So far, we assumed that threads communicate by reading and writing shared memory. This style naturally gives rise to mutual exclusion, condition variables, and races over shared state.

A different viewpoint is possible: *communicate to share* rather than *share memory to communicate*. In message-passing systems, threads do not jointly manipulate one shared variable structure. Instead, they exchange messages through queues or channels. The synchronization is expressed in the communication itself: if the channel is empty, a receiver waits; if a bounded channel is full, a sender waits.

Definition

In the **shared-memory style**, threads communicate by directly accessing common state and using locks or monitors to control the access discipline. In the **message-passing style**, threads communicate by explicitly sending data through channels or queues, often avoiding direct sharing of mutable state.

5.2 A Simple Go Example

Listing 6: Go Channels as a Message-Passing Synchronization Mechanism

```
func pinger(c chan string) {
    for {
        c <- "ping"
    }
}

func printer(c chan string) {
    for {
        msg := <-c
        fmt.Println(msg)
        time.Sleep(time.Second * 1)
    }
}

func main() {
    var c chan string = make(chan string)
    go pinger(c)
    go printer(c)
}
```

The channel here is effectively a queue plus a synchronization protocol. The sender and receiver do not need to share an explicit lock, shared counter, or condition variable. The channel abstraction carries both the data and the waiting semantics.

5.3 Equivalent Expressive Power, Different Programming Style

From a theoretical point of view, locks, semaphores, monitors, and channels can often implement one another. A semaphore can be viewed as a token queue; a queue can be implemented with locks and condition variables; a monitor can be emulated using message exchanges. The real difference is not expressive power but *what state is explicit* and *what invariants become easy to see*.

In shared-memory code, much of the application state remains distributed across ordinary variables, while the synchronization objects say relatively little. In message-passing code, the flow of state is often more explicit because the data moves through the channel itself.

Neither style is universally superior. Shared-memory code can be compact and familiar, especially in object-oriented systems. Message passing can be more verbose, but it often simplifies reasoning because ownership and state flow are more explicit. The central systems lesson is that synchronization is not tied to one notation; it is a design problem that can be expressed through multiple abstractions.

6 Concurrency Bugs in Practice

A large fraction of reported concurrency bugs fall into three categories:

1. **Atomicity violations**: operations that should have executed as one protected unit were not sufficiently synchronized.
2. **Order violations**: one thread incorrectly assumed that another event must already have happened.
3. **Deadlocks**: threads waited cyclically on one another.

The first category is often caused by *under-protection*: not enough synchronization. The third is often caused by *over-protection*: too many locks, or locks acquired in the wrong order. A recurring engineering tragedy is that a fix for a deadlock bug may weaken synchronization elsewhere and create an atomicity bug.

Another important observation is that many deadlocks in practice are structurally small: two threads and two resources are often enough. This does not make them easy to debug. On the contrary, concurrent bugs are notoriously expensive because they may depend on rare interleavings, disappear when additional logging changes the schedule, and resist deterministic reproduction.

Deep Dive

Concurrency bugs often consume disproportionate debugging time because the act of observing a schedule perturbs it. Adding logging, breakpoints, or instrumentation can change timing enough to hide the very behavior under investigation. This is one reason why simple synchronization disciplines are so valuable: prevention is far cheaper than post hoc diagnosis.

7 Resource Contention and Deadlocks

7.1 Resources: Preemptable and Non-Preemptable

A *resource* is any passive entity that a thread needs in order to make progress. CPU time, memory, disk space, locks, and exclusive device access are all examples, but not all resources behave alike.

- **Preemptable resources** can be taken away and reassigned without inherently breaking correctness. CPU time is the standard example because the operating system can context-switch away and resume later.
- **Non-preemptable resources** cannot be safely taken away in the middle of use without additional recovery machinery. A lock, a printer during printing, or a partially completed critical update fits this category.

The distinction matters because deadlock is fundamentally about resources that cannot simply be revoked at no cost.

7.2 Starvation versus Deadlock

These concepts are related but not identical.

- **Starvation** means a thread waits indefinitely, possibly because other threads keep being favored by the policy.
- **Deadlock** means a set of threads is stuck in a circular wait, each holding something the next needs.

A deadlocked thread is certainly starved, but a starved thread need not be in a deadlock. In starvation, the system may still be doing useful work for others. In deadlock, the blocked cycle cannot resolve itself without external intervention or rollback.

7.3 Deadlocks Are Often Nondeterministic

A classic two-lock example illustrates this point. Suppose thread A acquires x then y, while thread B acquires y then x. The program does *not* deadlock on every execution. Deadlock occurs only when the timing is unlucky enough that each thread acquires one lock before the other has a chance to acquire both. This is a crucial conceptual point: many deadlock questions ask whether deadlock *can* happen, not whether it *must* happen.

7.4 The Four Necessary Conditions

Deadlock requires four conditions to hold simultaneously. They are often known as the Coffman conditions.

Definition

The four necessary conditions for deadlock are:

1. **Mutual exclusion**: at least one required resource is non-sharable.
2. **Hold and wait**: a thread holds one resource while waiting for another.
3. **No preemption**: resources are released only voluntarily.
4. **Circular wait**: the wait-for relationships form a cycle.

To prevent deadlock, it is sufficient to break at least one of these conditions.

This framework is extremely useful because it transforms deadlock analysis from a vague intuition into a structured method. When presented with a complicated scenario, ask:

1. What exactly are the resources?
2. Which of them are exclusive?
3. Can a thread hold one while waiting for another?
4. Can anything be rolled back or forcibly revoked?
5. Is there a cycle in the wait-for structure?

7.5 Case Study: Bridge Crossing

The one-lane bridge example is instructive because it shows that resources are not always explicitly labeled as "lock x" and "lock y." Sometimes the analyst must identify them. Here the useful model is to treat bridge segments as resources. Each car owns its current segment and

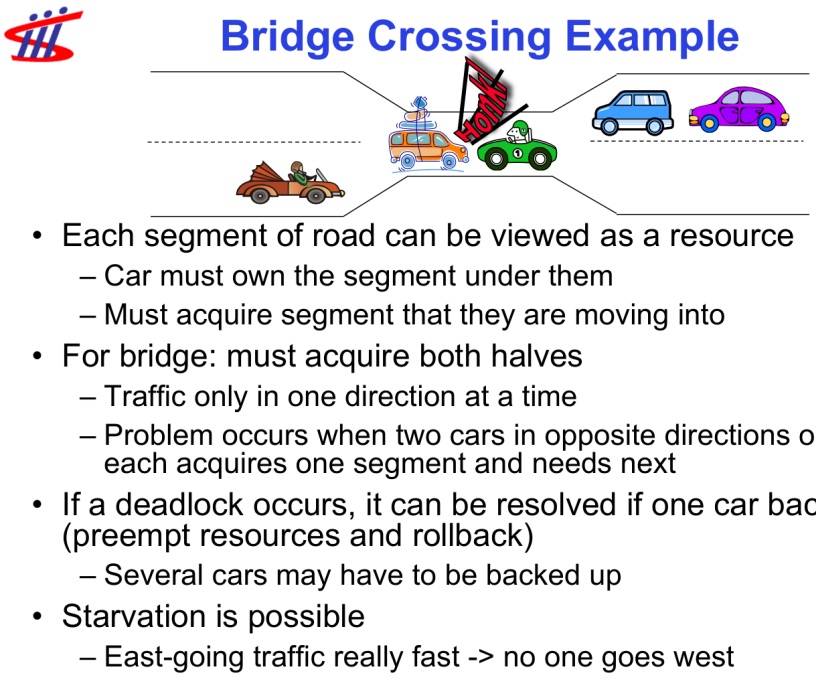


Figure 3: Bridge-crossing deadlock as a resource-allocation problem. Each road segment can be modeled as a resource. A car occupies the segment under it and must acquire the next segment before moving. When two cars enter a one-lane bridge from opposite directions, each may hold one segment while waiting for the next, creating hold-and-wait and circular-wait conditions.

must acquire the next segment before advancing. Two cars entering from opposite directions can each hold one segment while waiting for the other to release the next.

This example also clarifies recovery. If one car can back up, then a held resource is effectively preempted and rolled back, breaking the cycle. That is conceptually simple, but it may be operationally expensive, especially if additional cars have piled up behind the blocked ones.

7.6 Case Study: Dining Philosophers

The dining philosophers problem packages the deadlock structure into a clean graph. Each philosopher is a thread, each chopstick is a resource, and every philosopher needs two resources before proceeding. If all philosophers pick up their left chopstick first, every thread holds one resource and waits for another. The wait-for graph becomes a cycle of length five.

This example is especially useful pedagogically because it separates three questions:

1. **How can deadlock happen?** Everyone grabs one chopstick and waits.
2. **How could we recover?** Force someone to release a chopstick and retry.
3. **How can we prevent it?** Impose an asymmetric acquisition policy, a global ordering, or a protocol that prevents hold-and-wait from forming.

7.7 Detection and Recovery

One family of strategies allows the system to enter deadlock and then attempts to recover. Conceptually, the operating system or runtime constructs a resource-allocation or wait-for graph

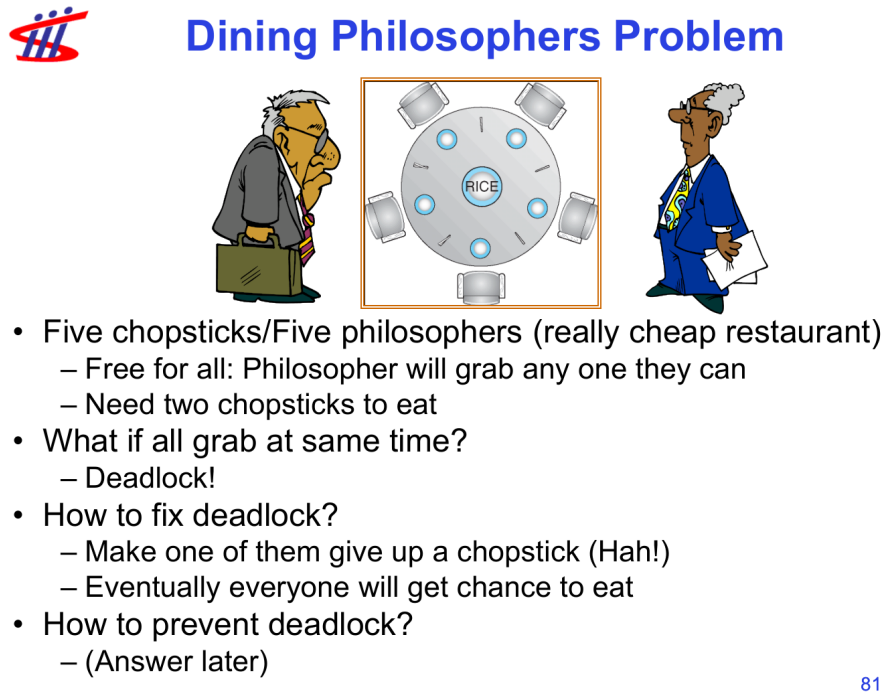


Figure 4: The dining philosophers problem. Each philosopher needs two chopsticks to eat, but the chopsticks are shared with neighbors. If every philosopher picks up one chopstick simultaneously and then waits for the second, the system forms a circular wait and deadlocks.

and checks for cycles. If a cycle is found, the system has identified a deadlock.

Detection is only the beginning. Recovery is difficult because a thread may have partially completed a larger operation. Killing one thread may release its locks, but it does not automatically restore the consistency of higher-level state. Databases address this with transactions and rollback: abort one transaction, undo its partial effects, and let the others continue. Without such semantics, recovery is often application-specific and expensive.

7.8 Prevention, Avoidance, and Practical Operating Systems

We distinguishes three broad approaches:

7.8.1 Prevention

Break one of the necessary conditions in advance. Common techniques include:

Technique	Condition Broken	Cost or Limitation
Make the resource effectively abundant	Mutual exclusion or scarcity pressure	Often unrealistic; may only work for some resources
Force request-all-at-once discipline	Hold and wait	Poor utilization; future needs are hard to predict
Allow rollback or forced release	No preemption	Requires recovery semantics
Impose global acquisition order	Circular wait	Constrains design and requires discipline everywhere

A global ordering is especially practical in software systems. If every thread acquires locks in the same order, circular wait becomes impossible because the resource graph has an acyclic ranking.

7.8.2 Avoidance

Instead of using a static rule, an avoidance algorithm simulates the consequences of granting a resource request and asks whether the resulting state is still *safe*. The classical model is the Banker's Algorithm. It is elegant but often impractical because it assumes knowledge of future maximum resource demands—knowledge that real operating systems generally do not possess.

7.8.3 Ignoring the Problem

This sounds irresponsible, but it is often the default operating-system posture for general application-level deadlock. The reason is semantic ignorance. The OS does not know whether killing a thread is acceptable, whether a partially completed update can be rolled back safely, or what higher-level resource requests the application may issue in the future. As a result, many operating systems simply provide the mechanisms (locks, wait queues, scheduling) and leave deadlock avoidance or diagnosis to the application or database system built on top.

Deep Dive

In practice, deadlock handling is easiest when the system also owns the semantics of recovery. Databases can abort transactions because a transaction is explicitly defined to be reversible until commit. General operating systems do not have that luxury for arbitrary user programs, which is why deadlock prevention and careful lock discipline remain central programming responsibilities.

8 Behind the Scenes: Implementing a Lock

This part is just intended to provide some visions and insights. Nevertheless, it is valuable because it connects synchronization abstractions to hardware and the operating system.

The central question is this: a lock's `Acquire()` operation is itself a multi-instruction activity. How can such an operation be made atomic?

8.1 The Naive Uniprocessor Idea: Disable Interrupts

On a single processor, preemption depends on interrupts. A naive idea is therefore to disable interrupts while a thread holds a lock and re-enable them on release. This would indeed prevent context switches inside the critical region.

But this is not acceptable as a general user-level mechanism. If arbitrary user code were allowed to disable interrupts and then loop forever, the operating system would lose control of the machine. Even without malicious behavior, ordinary user critical sections may be long or unpredictable, which would make response time and I/O handling intolerable.

8.2 A Better Interrupt-Based Design

The better idea is the same structural idea used earlier in the readers–writers solution: keep the *internal* synchronization critical section short. The system disables interrupts only while examining and updating the lock's internal metadata. If the lock is busy, it moves the thread onto a wait queue and puts it to sleep. The user's actual protected code does *not* execute with interrupts disabled.

This already reveals a deeper truth: blocking a thread and releasing internal protection must themselves be coordinated atomically. Otherwise the system risks lost wakeups or inconsistent scheduler state.

8.3 Atomic Read–Modify–Write Instructions

Interrupt disabling is insufficient on multiprocessors because disabling interrupts on one core does not stop another core from modifying the same memory. Modern systems therefore rely on hardware-supported atomic read–modify–write instructions.

Representative examples include:

- **test-and-set**: read a memory word and simultaneously set it to 1,
- **swap**: exchange a register value and a memory value atomically,
- **compare-and-swap**: replace a memory value only if it still equals an expected old value.

These instructions are implemented by hardware together with the cache-coherence protocol so that no two processors observe a non-atomic intermediate result.

8.4 A Simple Spinlock

Algorithm 3: Simple Spinlock Using test-and-set

Input: A lock word `value`, initially 0 for free
Output: Mutual exclusion with busy waiting

```
1 Acquire:;
2 while test&set(value) == 1 do
3     continue spinning;
4 Release:;
5 value = 0;
6 return success;
```

If the lock is free, `test&set` returns 0 and simultaneously writes 1, so the caller acquires the lock. If the lock is already busy, the instruction returns 1, and the caller loops.

This is correct but potentially wasteful. Threads waiting for the lock burn CPU cycles continuously. Such busy waiting is tolerable only when the expected hold time is very short, for example in extremely small kernel critical sections or on systems where sleeping would cost more than spinning briefly.

8.5 Reducing Busy Waiting

A more sophisticated lock uses a small atomic guard only to protect the lock's internal state and wait queue. If the main lock is busy, the thread is placed on a wait queue and put to sleep

instead of spinning indefinitely. In that design, busy waiting is confined to the very short period required to acquire the internal guard itself. The user's long critical section remains outside the spin.

This layered construction is conceptually important. High-level locks are often built from lower-level atomic instructions plus scheduler support. The expensive hardware primitive is used only for a tiny inner operation; larger waiting behavior is delegated to the operating system.

8.6 Comparison with the Interrupt-Based Solution

The correspondence is elegant:

- "disable interrupts" becomes "acquire an internal atomic guard,"
- "enable interrupts" becomes "release that guard,"
- sleeping and waking are coordinated through a wait queue maintained under the protection of the guard.

In other words, hardware atomics replace the single-processor assumption with a multiprocessor-safe mechanism for briefly protecting the lock's metadata.

9 Summary

This lecture deepens the study of concurrency by moving from primitive mutual exclusion to full synchronization design.

At the algorithmic level, the readers–writers problem shows how a monitor can express a policy richer than simple exclusion: many readers may proceed together, but writers require exclusivity. The solution's counters and condition variables are not arbitrary bookkeeping; they encode fairness, admission rules, and wakeup policy.

At the software-engineering level, the lecture shows why language support matters. Explicit locking is fragile in the presence of multiple control-flow paths and exceptions. C++ uses scope-based destructors to tie lock release to object lifetime, while Java builds monitors directly into the language through `synchronized` methods and blocks.

At the systems-design level, the lecture broadens the viewpoint from shared-memory synchronization to message passing and then to empirical observations about concurrency bugs. The most important practical warning is that complex synchronization written for marginal performance reasons often becomes a maintenance and debugging liability.

Finally, the lecture connects concurrency to resource contention and deadlock. Deadlock is best understood through resources, the four necessary conditions, and the structure of wait-for cycles. Preventing deadlock is often easier than recovering from it, especially when no rollback semantics exist. The concluding lock-implementation material reinforces a recurring theme of the course: small atomic mechanisms at the hardware and kernel level support much larger correctness guarantees at the software level.

10 Quizzes & Exercises

10.1 Short-Answer Quiz

Answer each question in two or three sentences.

1. What are the two primary reasons for using multiple concurrent threads in a system?
2. How does an atomic operation differ from a standard multi-step operation in a concurrent environment?
3. In the context of synchronization, what is a critical section, and what mechanism is used to implement it?
4. How does a semaphore function as a generalized lock compared to a standard binary lock?
5. What are the two distinct ways semaphores are typically used in thread collaboration?
6. Why is it generally safer to use a monitor rather than raw semaphores for complex synchronization problems?
7. What is the specific purpose of a condition variable within a monitor?
8. How does the synchronized destructor pattern in C++ assist in preventing synchronization errors?
9. Describe the fundamental difference between the shared-memory and message-passing styles of synchronization.
10. Why is optimizing too early a common cause of concurrency bugs?

10.2 Concise Answer Key

Q	Answer
1	Multiple threads help overlap I/O with computation and also provide cleaner program structure by separating ongoing tasks into independent execution contexts.
2	An atomic operation is indivisible, whereas an ordinary multi-step operation may expose intermediate state to other threads and therefore can be interrupted in ways that violate correctness.
3	A critical section is a region of code that manipulates shared state and must not be executed concurrently by conflicting threads. It is implemented through mutual exclusion, typically with a lock.
4	A binary lock represents only free versus busy, while a semaphore maintains a counter and therefore can encode multiple units of availability or more general scheduling conditions.
5	Semaphores are used both for mutual exclusion and for scheduling constraints such as counting full slots, empty slots, or available tokens.
6	Monitors are safer because they separate mutual exclusion from condition waiting, making the code structure clearer and allowing a thread to wait while atomically releasing the lock.
7	A condition variable provides a waiting queue for threads that cannot yet continue because some logical predicate is false; it is used with `wait`, `signal`, and `broadcast`.

8	The C++ synchronized-destructor or RAII pattern ties lock release to scope exit, so the lock is released automatically even if the function returns early or throws an exception.
9	In shared-memory synchronization, threads coordinate by directly accessing common variables protected by locks or monitors; in message passing, threads exchange data explicitly through channels or queues.
10	Premature optimization often produces unnecessarily intricate synchronization protocols whose hidden corner cases are hard to reason about, increasing the risk of deadlock, starvation, or atomicity errors.

10.3 Essay Questions

1. Analyze the historical shift from user-level threads to kernel-level threads. Why was the kernel once considered too slow, and what changed in system design to make kernel threads practical in modern systems?
2. Discuss the reader–writer priority dilemma. How does a writer-priority solution prevent writer starvation, and what performance trade-offs does it impose on reader-heavy workloads?
3. Compare C++ RAII-based synchronization with Java's `synchronized` keyword. Which design places more responsibility on the programmer, and which one provides stronger language-level guarantees?
4. Use the bridge, restroom, or philosopher analogies to explain the difference between starvation and deadlock. Why can one occur even when resource utilization remains high?
5. Examine the practical limits of deadlock prevention and avoidance. Why do many operating systems avoid solving arbitrary application-level deadlock automatically?

10.4 Suggested Analytical Exercises

1. Starting from $AR = WR = AW = WW = 0$, trace the state transitions for the arrival sequence R_1, W_1, R_2, W_2 under the writer-priority monitor solution.
2. Rewrite the readers–writers monitor so that readers ignore waiting writers. Identify a schedule under which a writer starves.
3. Suppose a monitor uses a single condition variable for both readers and writers. Construct a schedule showing why waking the wrong class of thread can cause the system to stall.
4. For each of the four necessary conditions for deadlock, give one concrete strategy that breaks it and one realistic cost of doing so.
5. Explain why a pure spinlock is a poor choice when the critical section may involve blocking I/O or long latency memory accesses.